



KHWAJA FAREED
UEIT
RAHIM YAR KHAN

Institute of
Computer Science

CORE COMPUTER SCIENCE MODULES

Compiled Course Content & Topic Coverage

BS Computer Science | Session 2019 – 2023

Institute of Computer Science
Khwaja Fareed University of Engineering & Information Technology
(KFUEIT)

This document combines the detailed content and topic coverage of core Computer Science modules taught during the BS Computer Science program (Session 2019 - 2023) at the Institute of Computer Science, KFUEIT.

The course contents presented in this document were primarily provided by the respective course instructors/faculty members. For modules where complete course outlines were not directly available from the instructors, the content was compiled from the relevant course outlines and teaching materials, including lecture slides and notes provided by the respective course instructors/faculty members.

Prepared by:

Muhammad Awais Jamil

BS Computer Science
Khwaja Fareed University of Engineering & Information Technology

Academic Session:
2019 – 2023





Programming Fundamentals + Lab (4 Credit hours)

Credit Hours:	3 + 1	Course Code	COSC-1101 COSC-1201	Instructor	Ms. Sameen Fatima
----------------------	-------	--------------------	------------------------	-------------------	-------------------

Objectives:

This course provides a comprehensive introduction to the core concepts of high-level programming, using the C++ language to focus on algorithmic thinking, structured programming, and the fundamental syntax required for advanced application development

Course Learning Outcomes:

- Students will be able to design, implement, and debug basic programs in C++ using structured programming principles.
- Students will demonstrate proficiency in fundamental programming constructs including data types, control flow structures, and functions.
- Students will be able to analyze simple problems and develop efficient algorithms using arrays and strings to manage data.

Course Contents:

- **Programming Fundamentals:** Problem-solving concepts, algorithms, pseudocode, flowcharts, program structure, syntax, compilation process, and basic C++ programming environment.
- **Data Types and Variables:** Primitive data types, constants, variable declaration and initialization, type casting, and input/output operations.
- **Operators and Expressions:** Arithmetic, relational, logical, assignment, increment/decrement, conditional and other fundamental C++ operators.
- **Control Flow:** Selection structures (if, if-else, nested if, switch) and repetition structures (while, do-while, for), including loop control statements.
- **Functions and Modularization:** Function declaration/definition, parameters and arguments, return values, pass-by-value, pass-by-reference, function overloading, scope rules, and modular program design.
- **Recursion:** Recursive functions, base and recursive cases, and applications of recursion.
- **Arrays and Strings:** One-dimensional and multidimensional arrays, array processing and passing arrays to functions, C-style strings, string manipulation and basic string functions.
- **Pointers and Memory Management:** Pointers, pointer arithmetic, addresses, references, dynamic memory allocation and deallocation.
- **Structures and Basic User-Defined Types:** Structures, records, and organization of related data.
- **File Handling:** File input/output, reading from and writing to files, and basic data persistence.
- **Problem Solving and Algorithms:** Basic algorithm implementation, searching, sorting, simple computational problems, and introduction to algorithm efficiency.
- **Complexity and Runtime Analysis:** Basic time and space complexity concepts and analysis of simple algorithms.
- **Debugging and Testing:** Syntax, logical and runtime errors, debugging techniques, tracing, testing, validation, and program correctness.
- **Introduction to Classes and Objects:** Basic classes and objects as a foundation for subsequent OOP.

Text Books:

1. C++ How to Program by *Paul Deitel & Harvey Deitel*.

Recommended Books:

1. C++ Primer by *Stanley B. Lippman et al*

Sr. No	Weekly	Topics
1	Week 1st	Introduction to Programming Paradigms, C++ Environment Setup, Compilation vs. Interpretation, Algorithms, Pseudocode, Flowcharts, Basic Program Structure
2	Week 2nd	C++ Syntax, Variables, Constants, Basic Data Types (int, float, char, bool), Input/Output and Type Casting
3	Week 3rd	Arithmetic, Relational, Logical Operators, Assignment, Increment/Decrement, Conditional Operators and Expressions
4	Week 4th	Selection Structures I: if, if-else, and Nested if-else Statements
5	Week 5th	Selection Structures II: switch, Statement and Conditional Operator
6	Week 6th	Repetition Structures I: while, do-while, for Loops
7	Week 7th	Repetition Structures II: Nested Loops, Loop Control (break, continue)
8	Week 8th	Functions I: Defining and Calling Functions, Prototypes, Libraries, Parameters and Return Values
9	Week 9th	Functions II: Pass by Value, Pass by Reference, Function Overloading, Recursion, Scope Rules
10	Week 10th	Arrays I: One-Dimensional Arrays, Array Processing, Searching and Basic Sorting
11	Week 11th	Arrays II: Multidimensional Arrays, Passing Arrays to Functions, Complexity and Runtime Analysis
12	Week 12th	Strings: C-style Strings, String Manipulation, Basic String Functions
13	Week 13th	Pointers: Addresses, Pointer Arithmetic, References, Dynamic Memory Allocation
14	Week 14th	Structures and User-Defined Data Types, Basic File Handling and File I/O
15	Week 15th	File Handling (Input/Output with Files), Debugging, Testing, Error Handling and Program Validation
16	Week 16th	Introduction to Classes and Objects (Basic OOP Concepts as a Foundation for Next Course)



Object Oriented Programming					
Credit Hours:	3	Course Code	COSC-1102	Instructor	Mr. Muiz-u-din
Objectives:					
This course introduces the fundamental concepts of object-oriented programming using Java. It focuses on designing software using classes and objects and applying core OOP principles such as encapsulation, inheritance, polymorphism, and abstraction.					
Course Learning Outcomes:					
- Students will be able to design and implement programs using classes and objects in Java. - Students will demonstrate understanding of core OOP concepts including encapsulation, inheritance, polymorphism, and abstraction. - Students will be able to use constructors, methods, and access modifiers effectively. - Students will develop modular, reusable, and efficient programs using Java. - Students will apply exception handling and file handling in Java applications.					
Course Contents:					
- Introduction to OOP and Java basics: JVM, syntax, modular program design, and program structure - Classes and Objects: Defining classes, creating objects, access modifiers - Methods and Constructors: Method overloading, constructors - Encapsulation and Abstraction: Data hiding and interfaces - Inheritance: Types of inheritance, code reusability, and reusable components - Polymorphism: Method overloading and method overriding - Interfaces and Abstract Classes: Abstraction and implementation of abstract behavior - Exception Handling: try-catch, finally, throw/throws - File Handling: Reading and writing files in Java - Arrays and Strings in Java					
Text Books:					
Java: How to Program by <i>Deitel & Deitel</i> - Head First Java by <i>Kathy Sierra & Bert Bates</i>					
Recommended Books:					
Effective Java by Joshua Bloch					

Weekly Plan

Sr. No	Weekly	Topics
1	Week 1st	Introduction to Java and OOP concepts
2	Week 2nd	Java syntax, variables, and data types
3	Week 3rd	Classes and objects
4	Week 4th	Methods and constructors
5	Week 5th	Encapsulation and access modifiers
6	Week 6th	Inheritance basics
7	Week 7th	Types of inheritance and implementation
8	Week 8th	Polymorphism (overloading & overriding)
9	Week 9th	Abstract classes and interfaces
10	Week 10th	Exception handling
11	Week 11th	Arrays and strings
12	Week 12th	File handling
13	Week 13th	Packages, Java libraries, modular program design, and reusable components
14	Week 14th	GUI basics
15	Week 15th	Project development
16	Week 16th	Final revision and evaluation



Course title: Object Oriented Programming-Lab					
Credit Hours	1	Course Code	COSC-1202	Instructor	Mr. Muizzud din
Objectives:					
This module focused on developing practical OOP proficiency by implementing advanced programming concepts (Encapsulation, Inheritance, Polymorphism) in Java, crucial for building modular and scalable software systems.					
Sr. No	Weekly	Topics			
1	Week 01-02	Java Environment Setup (JDK, IDE). Writing and compiling basic Java programs, understanding the main method			
2	Week 03-04	Classes and Objects Creation: Practical Implementation of Methods, Constructors, and Object Interaction			
3	Week 05	Encapsulation: Data protection using access modifiers (private, public) and getter/setter methods.			
4	Week 06	Arrays of Objects and Object initialization: Complex data structures handling			
5	Week 07-08	Creating sub-classes and super-classes, understanding method overriding.			
6	Week 09	Polymorphism: Run-time and compile-time polymorphism (Method Overloading).			
7	Week 10	Abstract Classes and Interfaces implementation: Designing templates for class behavior.			
8	Week 11-12	Exception Handling (Try-Catch-Finally)			
9	Week 13	Packages, Modular Program Design, File Input/Output (I/O) using Java streams			
10	Week 14	Reusable Components, Collections Framework basics: ArrayLists and HashMaps			
11	Week 15-16	Comprehensive OOP Project: Practical software implementation of a small system (Student record system) using Inheritance, Polymorphism, and Encapsulation			



Data Structures & Algorithms + Lab (4 Credit hours)

Credit Hours	3 + 1	Course Code	COSC-2101 COSC-2201	Instructor	Mr. Saqib Ubaid
---------------------	-------	--------------------	------------------------	-------------------	-----------------

Objectives:

This module focused on algorithmic problem solving, practical implementation and empirical analysis of complex data structures and fundamental algorithms in Java. Emphasis was placed on solving advanced computational problems, particularly in graph theory (Shortest Path, MST), and comparing algorithm efficiency.

Course Learning Outcomes (CLOs)

CLO 1:

Students will be able to explain and apply fundamental data structures, abstract data types, and algorithmic concepts, including arrays, linked lists, stacks, queues, trees, graphs, hash tables, determinism, termination, and asymptotic complexity.

CLO 2:

Students will be able to design and implement fundamental searching, sorting, tree, hashing, and graph algorithms, including BFS, DFS, Dijkstra's algorithm, and Minimum Spanning Tree algorithms, using appropriate data structures.

CLO 3:

Students will be able to analyze algorithm correctness, time and space complexity, runtime efficiency, and select suitable data structures and algorithms for practical computational problem-solving.

Course Content

Abstract Data Types (ADTs), arrays, linked lists, stacks, queues, recursion, searching and sorting algorithms, binary trees, BST and AVL trees, heaps and priority queues, hash tables and collision resolution, graph representations and traversal algorithms (BFS, DFS), shortest path and Minimum Spanning Tree algorithms (Dijkstra, Kruskal, Prim), algorithmic problem solving, determinism and termination, and basic time and space complexity, implementation, correctness, and runtime analysis.

Sr. No	Weekly	Topics
1	Week 01-02	Practical implementation of Abstract Data Types (ADTs) using Arrays/Vectors; basic data structure concepts, algorithm characteristics, determinism and termination, asymptotic analysis (Big-O, Big-Theta, Big-Omega), time and space complexity
2	Week 03	Implementation of Linked Lists (Singly, Doubly, and Circular): Node creation, insertion, deletion, traversal, searching, and dynamic memory management
3	Week 04	Implementation of Stacks and Queues using Arrays and Linked Lists; Circular Queues, basic Queue variations, applications including expression evaluation, recursion and function-call management
4	Week 05	Sorting Algorithms (1 of 6): Practical implementation of Bubble Sort and Selection Sort and comparative analysis

5	Week 06	Sorting Algorithms: Merge Sort, Quick Sort and Heap Sort; recursive and divide-and-conquer approaches, complexity analysis and comparison of sorting algorithms
6	Week 07	Advanced Sorting and Recursion: Merge Sort and Quick Sort; recursion, divide-and-conquer strategy, recurrence concepts, correctness and runtime analysis
7	Week 08	Trees: Binary Trees, tree terminology, node creation, recursive tree traversal (Preorder, Inorder, Postorder), level-order traversal and basic tree operations
Mid Term Exam		
8	Week 09	Binary Search Trees (BSTs) and AVL Trees: Search, insertion, deletion, tree traversal, balancing, rotations and height analysis
9	Week 10	Graph Data Structures: Directed and undirected graphs, weighted and unweighted graphs, vertices and edges, degree, paths and cycles, adjacency matrix and adjacency list representations
10	Week 11	Graph Traversal Algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS), traversal applications, connectivity and path finding
11	Week 12	Shortest Path Algorithms: Dijkstra's Algorithm, weighted graphs, min-heap and priority queue implementation, runtime analysis and applications
12	Week 13	Minimum Spanning Trees: Kruskal's Algorithm, Union-Find/Disjoint Set data structure, cycle detection, weighted graphs and runtime analysis
13	Week 14	Minimum Spanning Trees: Prim's Algorithm, priority queues/heaps, comparison with Kruskal's Algorithm, correctness and complexity analysis
14	Week 15	Hashing and Hash Tables: Hash functions, hash table operations, collision resolution techniques (chaining and open addressing), performance and load factor analysis
15	Week 16	Final Lab Project: Algorithmic problem solving, data structure selection, algorithm implementation, debugging/testing, correctness and runtime analysis
Final Term Exam		



Tentative Weekly Course Plan

Course Title	Design & Analysis of Algorithms	Instructor	Mr. Mutiullah Jamil
Course Code	COSC-3115	Credit Hour	3

Objective:

This course develops a strong foundation in the design, analysis, implementation, and evaluation of algorithms for computational problems. It covers asymptotic analysis, recurrence relations, major algorithmic paradigms, graph algorithms, optimization techniques, computational complexity, and practical performance evaluation. Students learn to design correct and efficient algorithms, analyze their time and space requirements, and select appropriate approaches for complex computational and data-driven problems.

Course Learning Outcomes:

CLO 1: Analysis: Mathematically analyze the time and space complexity of algorithms using asymptotic notation, recurrence relations, and appropriate efficiency measures.

CLO 2: Design: Design algorithms using major paradigms including Divide-and-Conquer, Greedy, Dynamic Programming, Backtracking, Branch-and-Bound, and Local Search to solve computational and optimization problems.

CLO 3: Implementation: Implement and evaluate fundamental and advanced algorithms, including sorting, searching, graph, shortest-path, and spanning-tree algorithms, while ensuring correctness and practical efficiency.

CLO 4: Complexity: Analyze computational problems using concepts of tractability, NP, NP-hardness, NP-completeness, reductions, approximation algorithms, and the limitations of efficient algorithmic solutions.

Course Contents

Algorithm design and analysis fundamentals, asymptotic notation and time/space complexity, recurrence relations and Master Theorem, algorithm correctness and proof techniques, searching and sorting algorithms, Divide-and-Conquer, Greedy algorithms, Dynamic Programming and optimization, Backtracking, Branch-and-Bound and Local Search, graph algorithms including BFS, DFS, topological sorting, shortest paths and Minimum Spanning Trees, hashing, B-Trees and disjoint-set structures, NP-completeness and reductions, approximation algorithms, and practical implementation, correctness, efficiency, and performance analysis.

Reference Books

- *Introduction to Algorithms* by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein (CLRS).
- *Algorithm Design* by Jon Kleinberg and Éva Tardos.

Sr. No	Weekly	Topics
1	Week 01-02	Algorithm definition, Role of algorithms in computing, Algorithm design and analysis, Growth of Functions, Asymptotic Notations and complexity, correctness and efficiency analysis
2	Week 03	Methods for solving recurrence relations: Substitution Method, Recursion Tree, and the Master Theorem
3	Week 04-05	Analysis and implementation of Insertion Sort, Merge Sort (Divide and Conquer), and Heapsort (using Priority Queues)
4	Week 06	Quicksort, Randomized Algorithms, and establishing the lower bounds for sorting
5	Week 07-08	Introduction to Greedy Strategy, Proof of Correctness, Activities Selection Problem, Huffman Coding, Minimum Spanning Trees (Prim's & Kruskal's)
6	Week 09-10	Principles of Dynamic Programming, optimization techniques, Optimal Substructure, Overlapping Subproblems, Examples: Matrix Chain Multiplication, Longest Common Subsequence (LCS), Knapsack Problem
7	Week 11	Graph Representations, Graph Traversal Techniques: Breadth-First Search (BFS) and Depth-First Search (DFS), Topological Sort
8	Week 12	Dijkstra's Algorithm, Bellman-Ford Algorithm, and its application in negative weight cycles
9	Week 13	The Floyd-Warshall Algorithm
10	Week 14	Advanced data structures for efficiency: B-Trees (overview), Hash Tables, and Disjoint Sets
11	Week 15	Decision Problems, Complexity Classes P and NP, NP-Completeness and Reductions
12	Week 16	Introduction to Approximation Algorithms (e.g., Vertex Cover), Local Search, Backtracking, Branch-and-Bound, and other appropriate algorithms relevant to Data Science.

Student Evaluation criteria:

Student Evaluation criteria:	5%
Surprise Test, Quizzes:	5%
Semester Project:	15%
Mid Term Paper:	25%
Final Term paper (Marks Distribution: 20 % from Mid Term and 80% from Final Term), i.e. 10 + 40=50:	50%
Total:	100%

Student Responsibilities:

A student must have attended at least 75% of the classes held in a course in order to be allowed to sit in the final examination. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beeper must be Turned off (Proper classroom decorum [behavior] adopts, Course outlines and calendars explain requirements and assignments. Students are also responsible for doing all assigned work on time. Students may prepare Sketchbook for taking notes and for references.



Khwaja Fareed University of Engineering & Information Technology

Rahim Yar Khan

Institute of Computer and Software Engineering

Course Outline

COSC-1104 Software Engineering (3 Credit Hours)

Pre-Requisite: N/A
Program: Computer Science
Total Lectures: 32
Total Tutorials: 0

Instructor: Mr. Afzal Hussain
Office: Faculty Hall, ICT Building, KFUEIT
Email: afzal.hussain@kfueit.edu.pk
Office Hours: 08:00am -12:30 pm (Wednesday)

Course Introduction

This course emphasizes understanding the principles, methodologies, and practices of modern software engineering. The objective of this course is to provide students with in-depth knowledge of software development lifecycles (SDLC, Agile/Scrum, Waterfall), requirements analysis, software design and architecture, object-oriented design, implementation and testing, debugging, version control, software quality, documentation and collaborative development. Students will gain practical skills to design, model, evaluate, and maintain high-quality software systems.

Course Contents

- 1. Introduction to Software Engineering & Lifecycles:**
The Concept of Software Engineering, Professional Ethics, Software development lifecycles (SDLC, Agile/Scrum, Waterfall), Incremental and Spiral Models.
- 2. Requirements Engineering:**
Requirements Elicitation, requirements analysis, Functional vs Non-Functional Requirements, Software Requirement Specification (SRS) Development.
- 3. Software Architecture & Design:**
Architectural Patterns, software design and architecture, Component Level Design, Architectural Styles, Interface Design.
- 4. Object-Oriented Design & System Modeling:**
object-oriented design Principles, Structural and Behavioral UML Modeling (Class Diagrams, Use Case Diagrams, Sequence Diagrams, Activity Diagrams).
- 5. Implementation, Testing & Debugging:**
Coding Standards, implementation and testing Strategies, Unit Testing, Integration Testing, System Testing, Black-Box and White-Box Testing, debugging Techniques.
- 6. Version Control & Collaborative Development:**
Source Code Management, Git and GitHub Workflows, Branching Strategies, version control, Code Reviews, collaborative development Practices.
- 7. Software Quality Assurance:**
Concepts of Verification and Validation, software quality Attributes, SQA Planning, Software Metrics, Code Inspections and Audits.
- 8. Documentation & Maintenance:**
Technical documentation, User Manuals, Software Maintenance, Legacy System Evolution, Re-Engineering.

Mapping of CLOs and PLOs			
Sr. No	Course Learning Outcomes	PLOs	Blooms Taxonomy
CLO_1	Understand software development lifecycles (SDLC, Agile/Scrum, Waterfall) and foundational software engineering principles.	PLO1	C2 (Understanding)
CLO_2	Perform systematic requirements analysis, system modeling, and draft software documentation.	PLO2	C3 (Applying)
CLO_3	Apply principles of software design and architecture and object-oriented design using standard UML diagrams.	PLO3	C3 (Applying)
CLO_4	Execute implementation and testing, debugging, version control, SQA for software quality, and engage in collaborative development.	PLO5	C4 (Analyzing)
Overall Grading Policy			
Assessment Items		Percentage	
Quizzes		8%	
Assignments		7%	
Semester Project		10%	
Midterm Exam		25%	
Final Exam		50%	
Text and Reference Books			
- Ian Sommerville: <i>Software Engineering</i> - Roger S. Pressman: <i>Software Engineering: A Practitioner's Approach</i> - Craig Larman: <i>Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design</i> - Martin Fowler: <i>UML Distilled: A Brief Guide to the Standard Object Modeling Language</i>			
Administrative Instructions			
- According to institute policy, 80% attendance is <i>mandatory</i> to appear in the final examination. - Project proposal, SRS, architecture design, and final report must be submitted as per announced format and within due date. - In any case, there will be no retake of (scheduled/surprise) quizzes. - For queries, kindly follow the office hours in order to avoid any inconvenience.			
Computer/LMS Usage			
Students will be required to use StarUML / Enterprise Architect for system modeling, Git & GitHub for version control, Visual Studio Code / IDEs for implementation, and Microsoft Word/PowerPoint for software documentation and project presentations			
Lecture Breakdown			
Week	Topics		
Week 1	Introduction to Software Engineering		

	• Evolution of Software Engineering & The Software Crisis • Professional Ethics in Software Development
Week 2	• Overview of Software development lifecycles (SDLC, Agile/Scrum, Waterfall) • Traditional Process Models: Waterfall, Prototyping, Spiral Model
Week 3	• Modern Agile Methodologies: Agile/Scrum Framework • Scrum Roles, Sprints, User Stories, and collaborative development
Week 4	• Requirements Engineering Process • Functional vs Non-Functional Requirements • Techniques for requirements analysis & Elicitation
Week 5	• Writing Software Requirement Specification (SRS) • Validation and Verification of Requirements • Requirements Traceability & Software documentation
Week 6	• Fundamentals of software design and architecture • Architectural Styles: Layered, Client-Server, MVC, Microservices
Week 7	• Object-oriented design Principles & Concepts • SOLID Design Principles (Single Responsibility, Open-Closed, etc.)
Week 8	• System Modeling with Unified Modeling Language (UML) • Structural Diagrams: Class Diagrams & Component Diagrams
Week 9	Mid Term Exam
Week 10	• Behavioral UML Diagrams: Use Case, Sequence, and Activity Diagrams • Mapping Requirements to Design Artifacts
Week 11	• Implementation and testing Strategies • Coding Standards and Refactoring • Version control with Git/GitHub and Branching Workflows
Week 12	• System Defect Tracking • Practical debugging Techniques & Strategies
Week 13	• Software Testing Levels: Unit, Integration, System, Acceptance Testing • Black-Box vs White-Box Testing Techniques
Week 14	• Software Quality Assurance (SQA) Concepts • Managing software quality Attributes and Software Metrics
Week 15	• Software Technical documentation Standards • Software Maintenance Types: Corrective, Adaptive, Perfective
Week 16	• collaborative development Project Submissions • Semester Project Code Reviews & Demos
Week 17	• Final Project Presentations & Evaluations
Week 18	Final Exam



Tentative Weekly Course Plan

Course Title	Database Systems + Lab (4 Credit hours)		
Course Code	COSC-2103 and COSC-2203	Credit Hour	3 + 1
Name	Mr. Adeel Abid	Email	adeel.abid@kfueit.edu.pk

Objective

This course is to provide students with a comprehensive understanding of the principles, design, and management of Database Management Systems (DBMS). The course focuses on fundamental data models, especially the Relational Model, query languages, and the principles of database integrity and concurrency

Course Learning Outcomes (CLOs)

CLO 1: Data Modeling: Design and develop robust and normalized database schemas using the Entity-Relationship (E-R) Model and map them to the Relational Model.

CLO 2: Querying & Manipulation: Write complex and efficient queries using Structured Query Language (SQL), including joins, sub-queries, and data manipulation commands (DML, DDL).

CLO 3: Normalization: Apply Normalization Theory (up to 3NF and BCNF) to eliminate data redundancy and anomalies, ensuring data integrity and consistency.

CLO 4: Transaction & Control: Explain the concepts of database transactions, concurrency control (e.g., locking), recovery mechanisms, and the basic principles of database security.

Course Content

Database architecture and ANSI-SPARC architecture, data independence, database design and ER/EER modeling, ER-to-relational mapping, relational model and relational algebra, relational calculus, keys and integrity constraints, SQL including DDL, DML, joins, nested queries, aggregate functions, views, stored procedures, functions and triggers, functional dependencies and normalization from 1NF to BCNF, lossless decomposition, query processing and optimization, transaction management and ACID properties, concurrency control and deadlocks, recovery techniques, physical storage and file organization, indexing using B-Trees/B+ Trees, hierarchical, network and object-oriented database models, NoSQL and data warehousing concepts, database security and authorization, and practical database implementation and SQL laboratory exercises.

1. Reference Books

i *Database System Concepts by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan*

ii *Fundamentals of Database Systems by Ramez Elmasri and Shamkant B. Navathe*

2. Web Resources

i. w3school.com

ii. geeksforgeeks.org/dbms

Week #	Topics (outline of main topics and sub topics)
1 - 2	Database Architecture, ANSI-SPARC Architecture, Data Independence, Database Users and DBMS vs File Systems
3 - 4	Relational Model Concepts, Keys (Primary, Foreign, Candidate), Integrity Constraints, Relational Algebra Operations, and Relational Calculus
5 - 6	Entity-Relationship (ER) Modeling, ER Components (Entities, Attributes, Relationships), Enhanced ER (EER) Modeling, and ER-to-Relational Schema Mapping
7 - 8	SQL DDL/DML, Joins, Aggregate Functions, GROUP BY/HAVING, Nested Queries, Views
Mid Term Exam	
9 - 10	Advanced SQL, Set Operations, Stored Procedures, Functions and Triggers
11 - 12	Functional Dependencies, Normalization Forms (1NF, 2NF, 3NF, BCNF), Lossless Join Decomposition
13	Concept of Transactions, ACID Properties (Atomicity, Consistency, Isolation, Durability), Transaction States
14	Concurrency Control, Lock-Based Protocols, Deadlocks and Timestamp Ordering
15	Physical Storage, File Organization, Indexing, B-Trees/B+ Trees, Recovery and Backup
16	Hierarchical, Network and Object-Oriented Database Models, NoSQL, Data Warehousing, Database Security and Final Database Lab Project
Final Term Exam	

Assignments	5%
Surprise Test, Quizzes	5%
Presentation	15%
Mid Term Paper	25%
Final Term paper (Marks Distribution: 20 % from Mid Term and 80% from Final Term), i.e. 10 + 40=50	50%
Total	100%

Student Responsibilities:

A student must have attended at least **75%** of the classes held in a course in order to be allowed to sit in the final examination. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beeper must be Turned off (Proper classroom decorum [behavior] adopts, Course outlines and calendars explain requirements and assignments. Students are also responsible for doing all assigned work on time. Students may prepare Sketchbook for taking notes and for references.



Department of Computer Science

Tentative Weekly Course Plan

Course Title	Web Programming		
Course Code	COSC-4120	Credit Hour	3

Course Learning Outcomes (CLOs):

1. Understand Web technologies and client-server architecture along with the core mechanics of HTTP.
2. Develop front-end user interfaces with HTML/CSS and Javascript for interactive dynamic web programming.
3. Implement server-side processing, forms and data handling, and persistent database integration.
4. Build, configure, and execute the end-to-end deployment of web-based applications.

Resource Material	1. Reference Books	
	i	Jon Duckett: <i>HTML and CSS: Design and Build Websites</i>
	ii	Jon Duckett: <i>JavaScript and JQuery: Interactive Front-End Web Development</i>
	iii	Robin Nixon: <i>Learning PHP, MySQL & JavaScript</i>
	2. Web Resources	
	i.	https://www.w3schools.com/
	ii.	https://www.freecodecamp.org/news/learn-web-development-for-free/

TOPICS TO BE COVERED

Week #	Topics (outline of main topics and sub topics)	Tutorial /Laboratory
1	• Introduction to Web technologies and client-server architecture • Evolution of the web and modern standards	
2	• Mechanics of HTTP request and response flow • Client-side vs server-side roles	Quiz
3	• Document structure and semantic elements using HTML/CSS • Standard web page formatting ➤ Assignment to achieve CLO 1	
4	• Styling pages and responsive grid design with HTML/CSS • Media queries and page layouts	
5	• Introduction to dynamic web programming using Javascript • Basic syntax, variables, and control flow	



Department of Computer Science

6	- Interactive DOM manipulation with Javascript - Dynamic element generation	
7	- Event handling and client-side page interaction - Input field monitoring ➤ Assignment to achieve CLO 2	
8	- Building web user interface forms and data handling - Client-side data validation	Quiz
Mid Term Exam		
10	- Fundamentals of server-side processing logic - Setting up backend execution environments	
11	- Routing requests and server-side processing workflows - Session and cookie management	
12	- Database integration principles for web applications - Connecting web servers to persistent storage ➤ Assignment to achieve CLO 3	
13	- Executing database operations and queries - Processing backend form data into databases	Quiz
14	- Web security fundamentals, input sanitization, and session safety - Application protection basics	
15	- Full-stack web application development consolidation - Preparation for deployment of web-based applications ➤ Assignment to achieve CLO 4	
16	Live web application project testing and reviews	Quiz
17	Final project presentations	
Final Term Exam		

Student Evaluation criteria:

Workshop / Assignments/Case study	5%
Surprise Test, Quizzes	5%
Semester Project	15%
Mid Term Paper	25%
Final Term paper	50%
Total	100%



KHWAJA FAREED
UEIT
RAHIM YAR KHAN

Faculty of
**Information Technology &
Management Sciences**

Department of Computer Science

Instructor:

Mr. Maqsood Ahmad

Email: maqsood.ahmad@kfueit.edu.pk

Consulting hours: 12:00pm-02:00pm Monday-thursday.

Office: 2.03L, Department of Computer Science, ICT Building, KFUEIT

Office Hours (Instructor):

Monday - Thursday: 08:00 – 16:30

Student Responsibilities:

Students must attend class. Failure to attend class may result in failure in the course. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beeper must be Turned off (Proper classroom decorum [behavior] adopts, Course outlines and calendars explain requirements and assignments, students are responsible for knowing what they say. Students are also responsible for doing all assigned work on time. Excessive absences (more than 03) will result in “F Grade”. Students may prepare Sketchbook for taking notes and for references.

Instructor/ Subject Specialist

Chairman/ Head of Department



Khawaja Fareed University of Engineering & IT Rahim Yar Khan

Tentative Course Plan **Department of Computer Science**

Class:

Semester:

Session:

Instructor:	Ms. Humaira Anwer	Email: humaira.anwer@kfueit.edu.pk
Course Title:	Artificial Intelligence + Lab	Program: BSCS
Course Number:	COSC-3112 and COSC-3212	Credit Hours: (3+1) = 4

Lecture Time:	
<u>Course Objective & Outcomes:</u> - Understand key components in the field of artificial intelligence - Analyze artificial intelligence techniques for practical problem solving - Install/Establish Implement classical artificial intelligence techniques	
<u>Methods of Teaching:</u> - Class participation is very important and will be evaluated on the bases of class attendance, punctuality, ability to contribute in class discussions, participation in answer questions session and preparation level with assigned readings. All students are expected to prepare for and participate in class. - The methodology of Direct / Live teaching at class room as per Lecture schedule where student have the freedom to interact and ask any question to teacher regarding the current Lecture topic. - Student would also have option to contact with teacher at specific office hours (may change according to summer and winter season) regarding the course related queries.	
Resource Material	1. Reference Book
	i Artificial Intelligence. A Modern Approach, Stuart Russell and Peter Norvig, 4 th Edition, Prentice Hall, Inc., 2020; ISBN-10: 0134610997
	ii Artificial Intelligence with Python, Your Complete Guide to building Intelligent apps using Python 3.x and TensorFlow 2, Alberto Artasanchez, Prateek Joshi, 2 nd Edition, 2020 ISBN-10: 178646439X
	2. Web Resources
	i. https://www.aialab.com.au/resources/online-resources/
	ii. https://deeplearning.mit.edu/
Office Help Hours	During Official Hours

SEQUENCE OF TOPICS TO BE COVERED		
Week	Topics (outline of main topics and sub topics)	Book / Tutorial
1	Introduction to Artificial Intelligence: • What Is AI? • The Foundations of Artificial Intelligence • The History of Artificial Intelligence • The State of the Art • Risks and Benefits of AI	Chapter No. 1
2	Intelligent Agents: • Agents and Environments • Good Behavior: The Concept of Rationality • The Nature of Environments • The Structure of Agents	Chapter No. 2
3	Problem Solving By Searching: • Problem-Solving Agents • Example Problems • Search Algorithms • Uninformed Search Strategies	Chapter No. 3
4	Problem Solving By Searching: • Uninformed Search Strategies • Informed (Heuristic) Search Strategies • Heuristic Functions	Chapter No. 3
5	Adversarial Search and Games: • Game Theory • Optimal Decisions in Games • Heuristic Alpha--Beta Tree Search • Monte Carlo Tree Search • Stochastic Games • Partially Observable Games • Limitations of Game Search Algorithms	Chapter No. 5
6	Constraint Satisfaction Problems: • Defining Constraint Satisfaction Problems • Constraint Propagation: Inference in CSPs • Backtracking Search for CSPs • Local Search for CSPs • The Structure of Problems	Chapter No. 6
7	Logical Agents: • Knowledge-Based Agents • The Wumpus World - Explanation • Logic • Propositional Logic: A Very Simple Logic • Propositional Theorem Proving • Effective Propositional Model Checking • Agents Based on Propositional Logic	Chapter No. 7

8	First Order Logic & Knowledge Representation: • Representation Revisited • Syntax and Semantics of First-Order Logic • Using First-Order Logic • Knowledge Engineering in First-Order Logic • Ontologies and Knowledge Graphs	Chapter No. 8
9	Inference in First Order Logic & Knowledge Systems: • Propositional vs First-Order Inference • Unification and First-Order Inference • Forward Chaining • Backward Chaining • Rule-based and Expert Systems • Resolution & Automated Theorem Proving	Chapter No. 9
10	Learning: Automated Planning & Multiagent Decision Making: • Automated Planning • Properties of Multiagent Environments • Non-Cooperative Game Theory • Cooperative Game Theory • Making Collective Decisions	Chapter No. 18
11	Discrete Data Analysis & Learning from Examples: • Discrete Data Analysis • Forms of Learning • Supervised Learning • Learning Decision Trees • Model Selection and Optimization • The Theory of Learning	Chapter No. 19
12	Learning from Examples: • Linear Regression and Classification • Nonparametric Models • Ensemble Learning • Developing Machine Learning Systems	Chapter No. 19
13	Deep Learning: • Simple Feedforward Networks • Computation Graphs for Deep Learning • Convolutional Networks • Learning Algorithms • Generalization • Recurrent Neural Networks • Unsupervised Learning and Transfer Learning • Applications	Chapter No. 21
14	Natural Language Processing: • Language Models and Grammar • Parsing and Augmented Grammars • Complications of Real Natural Language • Natural Language Tasks	Chapter No. 23

15	Deep Learning for Natural Language Processing: • Limitations of Feature-Based NLP Models • Word Embeddings • Recurrent Neural Networks • Sequence-to-sequence Models • The Transformer Architecture • Pretraining and Transfer Learning	Chapter No. 24
16	Reinforcement Learning • Learning from Rewards • Passive Reinforcement Learning • Active Reinforcement Learning • Safe Exploration • Generalization in Reinforcement Learning • Policy Search • Applications of Reinforcement Learning	Chapter No. 22
Final Term Exam		

Student Evaluation criteria:

Workshop / Assignments/Case study	5%
Surprise Test/Sudden Test , Quizzes	5%
Semester Project	15%
Mid Term Paper	25%
Final Term paper	50%
Total	100%

Student Responsibilities:

Students must attend class. Failure to attend class may result in failure in the course. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beepers must be Turned off (Proper classroom decorum [behavior] adopted. Course outlines and calendars explain requirements and assignments, students are responsible for knowing what they say. Students are also responsible for doing all assigned work on time. Excessive absences (more than 03) will result in “F Grade”. Students may prepare Sketchbook for taking notes and for references.

Instructor/Tutor

Approved by:

Chairman/ HOD/ Subject Specialist



Course Title: Data Science					
Credit Hours:	3	Course Code	COSC-3109	Instructor	Mr. Maqsood Ahmad
Objectives:					
This course introduces the systematic process of Extracting Knowledge from Data. It emphasizes the foundational steps of the Data Science workflow, including data acquisition, cleaning, statistical analysis, and visualization. The course provides a strong statistical and computational background for handling and interpreting complex datasets, serving as a necessary precursor to advanced Machine Learning studies.					
Course Learning Outcomes:					
• CLO 1: Data Acquisition and Wrangling: Implement practical techniques for data integration, cleaning, and transformation using programmatic tools (e.g., Python/R libraries). • CLO 2: Statistical Foundation: Apply fundamental descriptive and inferential statistics (e.g., probability distributions, hypothesis testing) to explore and summarize data effectively. • CLO 3: Data Visualization: Select and create effective data visualizations (e.g., histograms, scatter plots, heatmaps) to uncover patterns and communicate insights. • CLO 4: Predictive Basics: Understand and implement basic linear modeling concepts and evaluate the quality of simple predictive models.					
Text Books:					
1. Doing Data Science by <i>Cathy O'Neil and Rachel Schutt</i> 2. R for Data Science by <i>Hadley Wickham and Garrett Golemund</i>					
Web resources:					
i.	https://kaggle.com/learn				
ii.	https://jakevdp.github.io/PythonDataScienceHandbook				
iii.	https://scikit-learn.org				

Weekly Plan of “Data Science” by Mr. Maqsood Ahmad

Sr. No	Weekly	Topics
1	Week 1-2	Data Science pipeline overview, Role of statistics, Data sources and types, data acquisition and preprocessing, Introduction to data manipulation tools with Python (NumPy, Pandas) and R
2	Week 3-4	Data cleaning and missing-value handling (Imputation), Outlier detection, feature engineering (Categorical Encoding, One-Hot Encoding), scaling and normalization, Data integration techniques
3	Week 5-6	Exploratory data analysis (EDA), statistical summaries (Mean, Median, Variance), Data summarization, Measures of central tendency and dispersion
4	Week 7-8	Probability Distributions (Normal, Binomial), Central Limit Theorem, Confidence Intervals, Introduction to Hypothesis Testing
Mid Term Exam		
5	Week 9-10	Principles of effective visualization using Python (Matplotlib, Seaborn), Univariate and Bivariate visualizations, plotting time series data, Creating dashboards (basic)
6	Week 11-12	Review of relational data concepts, Basic SQL queries for data extraction and joining tables, Connecting to data sources
7	Week 13-14	Introduction to Machine Learning with Scikit-Learn: Simple Linear Regression, Clustering (K-Means), Dimensionality Reduction (PCA), Model Evaluation & Validation Metrics (MSE, R ² Score, Confusion Matrix, Silhouette Score)
8	Week 15	Introduction to concepts like Bayes' theorem, Basic decision rules (not algorithms like k-NN/SVM)
9	Week 16	Applying the entire workflow (from raw data to final report) on a real-world dataset
Final Term Exam		



Khawaja Fareed University of Engineering & IT Rahim Yar Khan

Tentative Course Plan **Department of Computer Science**

Class:

Semester:

Session:

Instructor:	Mr. Shahzad Hussain	Email:	shahzad.hussain@kfueit.edu.pk
Course Title:	Machine Learning	Program:	BSCS
Course Number:	COSC-3107	Credit Hours:	3

Lecture Time:	
<u>Course Objective & Outcomes:</u> 1. To introduce students to the basic concepts and techniques of Machine Learning. 2. To develop skills of using recent machine learning software for solving practical problems. 3. To gain experience of doing independent study and research.	
<u>Methods of Teaching:</u> • Class participation is very important and will be evaluated on the bases of class attendance, punctuality, ability to contribute in class discussions, participation in answer questions session and preparation level with assigned readings. All students are expected to prepare for and participate in class. • The methodology of Direct / Live teaching at class room as per Lecture schedule where student have the freedom to interact and ask any question to teacher regarding the current Lecture topic. • Student would also have option to contact with teacher at specific office hours (may change according to summer and winter season) regarding the course related queries.	
Resource Material	1. Recommended & Reference Books
	i Fundamentals of Machine Learning for Predictive Data Analytics, Algorithms, worked examples and case studies 2020, ISBN-10: 0262029448
	ii Practical Machine Learning with Python, 2017; ISBN-10: 1484232062
	iii Introduction to Machine Learning with Python, 2018 ISBN-10: 1449369413
	2. Web Resources
	i. https://machinelearningmastery.com/start-here/
	ii. https://medium.com/analytics-vidhya
Office Help Hours	During Official Hours

SEQUENCE OF TOPICS TO BE COVERED		
Week	Topics (outline of main topics and sub topics)	Book / Tutorial
1	Machine Learning and Applications: Introduction to Machine Learning and Applications: Why you should learn machine learning with us; who this specialization is for and what you will be able to do; getting started with the tools for the course; getting started with Python and Jupyter notebook; getting started with SFrames for data engineering and analysis	
2	Machine Learning Methods Supervised Learning: Classification; Regression. Un-supervised Learning: Clustering, Dimensionality reduction, Anomaly detection, Association rule-mining; Semi-Supervised Learning and Reinforcement Learning. Batch Learning, Online Learning, Instance Based Learning and Model Base Learning.	
3	Machine Learning and Real-world Applications (Use-Case Classification): Analyzing the Sentiment of Reviews; what is intelligent restaurant Review System; Examples of Classification tasks; Linear classifiers; Decision Boundary Evaluating of Classification: Training and evaluating a classifier; model evaluation (Cross-validation, Confusion Matrix, Precision, Recall, ROC-AUC); What a good accuracy; False Positive, false negative; Learning curves; Class prediction Assignment: To implement various Supervised Learning Machine Learning Models on same dataset and for the same task.	
4	Machine Learning and Real-world Applications (Use-Case Regression): Predicting a house price; what good and how might you naively address it? A linear regression: a model-based approach; adding higher order effects. Evaluating regression models: Evaluating overfitting via training / test split; Training / test curves; adding their features; Other regression examples Assignment: To implement various Supervised Machine Learning Regression Models on same dataset and for this task.	
5	The CRISP-DM Process Model Business Understanding, Define Business Problem, Assess and Analysis Scenarios, Define the data-mining problem, Project plan, Data Understanding, Data Collection, Data Description, Exploratory Data Analysis, Data Quality Analysis, Data Preparation, Data Integration, Data Wrangling, Attribute Generation and Selection Implementation CRISP-DM Process Model in real world problem: Real-World Case Study: Predicting Student Grant Recommendations	
6	Processing, Wrangling and Visualizing Data: Data Collection: CSV, JSON, XML, HTML and Scraping, HTML, Web Scraping, SQL, Data Description: Numeric, Text, Categorical Data Wrangling: Understanding Data, Filtering Data, Typecasting, Transformations, Imputing Missing Values, Handling Duplications, Handling Categorical Data, Normalizing Values, String	

	Manipulations.	
7	Processing, Wrangling and Visualizing Data: Data Summarization: Data Visualization: Visualizing with Pandas, Line Charts, Bar Plots, Histograms, Pie Charts, Box Plots, Scatter Plots, Visualization with Matplotlib, Figures and Subplots, Plot formatting, Legends, Axis Controls, Annotations, Global Parameters, Python Visualization Ecosystem.	
8	Data Preprocessing & Feature Engineering: Data and Datasets, Features, Models; Feature Engineering on Numeric Data, Feature Engineering on Categorical Data. Feature Engineering on Text Data, Feature Engineering on Temporal Data, Feature Engineering on Image Data	
Mid Exam		
9	Dimensionality Reduction (PCA & Feature Selection): Standardized Scaling, Min-Max Scaling, Robust Scaling, Feature Selection, Threshold-Based Methods, Statistical Methods, Recursive Feature Elimination, Model-Based Selection, Dimensionality Reduction: Feature Extraction with Principal Component Analysis,	
10	Building, Tuning and Deploying Models: Supervised Learning: Classification and Regression; Generalization, Overfitting and Underfitting; Supervised Machine Learning Algorithms; k-Nearest Neighbors Model: k-Neighbors classification, analyzing KneighboursClassifier, k-neighbors regression, analyzing k-nearest neighbors region, strength, weaknesses and parameters Linear Models: Linear Models: Linear Model for regression and classification; Linear/Logistic Regression; Ridge regression; Lasso; regularization; Linear models for multi-class classification, strength, weaknesses and parameters.	
11	Building, Tuning and Deploying Models: Supervised Learning: Decision Trees: Building Decision Trees; Controlling complexity of Decision Trees & Probabilistic Models: Naive Bayes classification; Building Decision Trees; Controlling complexity of Decision Trees; Analyzing Decision Trees; Feature Importance in trees; Strength, weaknesses and parameters, Ensembles of Decision Trees; Random Forests and Gradient Boosted Regression Trees (GBM). Kernelized Support Vector Machines: Linear Models and Non-linear features, the kernel trick, understanding SVMs, tuning SVM parameters, Preprocessing data for SVMs, strength, weakness and parameters	
12	Building, Tuning and Deploying Models: Supervised Learning: Neural Networks (Deep Learning): the neural network model, tuning neural networks, strength, weaknesses and parameters Classifying movie reviews: a binary classification example	

13	Building, Tuning and Deploying Models: Un-Supervised Learning: Types of unsupervised learning; Preprocessing and Scaling; Dimensionality Reduction, Feature Extraction and Manifold Learning, PCA, NMF, Manifold learning with t-SNE	
14	Building, Tuning and Deploying Models: Un-Supervised Learning: Clustering: k-Means clustering, agglomerative clustering, DBSCAN, comparing and evaluating clustering algorithms; evaluating clustering with ground truth.	
15	Real World Case Studies: Analyzing Bike sharing Trends The Bike sharing Dataset; Problem Statement; Exploratory and Data Analysis; Regression Analysis; Modeling; Testing; Next Steps	
16	Real World Case Studies: Galaxy Classification Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, Deployment	
	Final Term Exam	

Student Evaluation criteria:

Workshop / Assignments/Case study	5%
Surprise Test/Sudden Test , Quizzes	5%
Semester Project	15%
Mid Term Paper	25%
Final Term paper	50%
Total	100%

Student Responsibilities:

Students must attend class. Failure to attend class may result in failure in the course. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beepers must be Turned off (Proper classroom decorum [behavior] adopted. Course outlines and calendars explain requirements and assignments; students are responsible for knowing what they say. Students are also responsible for doing all assigned work on time. Excessive absences (more than 03) will result in “F Grade”. Students may prepare Sketchbook for taking notes and for references.



Department of Computer Science

Tentative Weekly Course Plan

Course Title	Introduction to NLP		
Course Code	COSC-3121	Credit Hour	3

Course Learning Outcomes (CLOs):

1. Understand core concepts, text preprocessing methods, and language modeling techniques in Natural Language Processing.
2. Analyze and implement algorithms for POS tagging, syntactic parsing, and word embeddings.
3. Design and evaluate deep learning and statistical models for text classification and sentiment analysis.

Resource Material	1. Reference Books	
	i	Daniel Jurafsky & James H. Martin: <i>Speech and Language Processing</i> (3rd Edition Draft)
	ii	Christopher D. Manning & Hinrich Schütze: <i>Foundations of Statistical Natural Language Processing</i>
	iii	Steven Bird, Ewan Klein, & Edward Loper: <i>Natural Language Processing with Python (NLTK)</i> (O'Reilly)
	2. Web Resources	
	i.	https://www.nltk.org/book/
	ii.	https://web.stanford.edu/class/cs224n/

TOPICS TO BE COVERED

Week #	Topics (outline of main topics and sub topics)	Tutorial /Laboratory
1	• Introduction to NLP • Language Thoughts	
2	• Regular Expression and Automata • Text Normalization & Preprocessing (Tokenization, Stemming, Lemmatization)	Quiz
3	• Minimum Edit Distance • Backtrace for Computing Alignments ➤ Assignment to achieve CLO 1	
4	• N-gram Language Models • Evaluation and Perplexity	
5	• Lexical Classes & POS	



Department of Computer Science

	• POS Tagger and Practice	
6	• Text Representation & Feature Weighting • Term Frequency-Inverse Document Frequency (TF-IDF)	
7	• Distributed Representation & Word Embeddings • Word2Vec (CBOW, Skip-Gram) and GloVe Assignment to achieve CLO 2	
8	• Text Classification Pipelines • Sentiment Analysis Foundations & Practice • Quiz 1	Quiz
Mid Term Exam		
10	• Deep Learning for Text Processing • Recurrent Neural Networks (RNN/LSTM) for Text	
11	• Constituency Grammars • Context-Free Grammars (CFG) & Sentence Construction Rules • Syntactic Parsing (CFG)	
12	• Sentence Construction Rules & Treebank • CKY Parsing Algorithm ➤ Assignment to achieve CLO 3	
13	• Probabilistic Context-Free Grammars (PCFGs) • Probabilistic CKY Algorithm	Quiz
14	• Logical Representation of Sentence Meaning • First-Order Logic for Computational Semantics	
15	• Syntax-driven Semantic Analysis • Semantic Parsing & Meaning Extraction Assignment to achieve CLO 4	
16	• Sequence-to-Sequence Architecture & Attention Mechanisms • Modern NLP Applications & Hands-on Framework Practice	Quiz
17	• Semester Project Presentations & Review	
Final Term Exam		

Student Evaluation criteria:

Workshop / Assignments/Case study	5%
Surprise Test, Quizzes	5%
Semester Project	15%
Mid Term Paper	25%



Department of Computer Science

Final Term paper	50%
Total	100%

Instructor:

Ms. Humaira Anwer

Email: humaira.anwer@kfueit.edu.pk

Office: Faculty Hall, Institute of Computer Science, ICT Building, KFUEIT

Office Hours (Instructor):

Monday - Thursday: 11:00 – 13:30

Student Responsibilities:

Students must attend class. Failure to attend class may result in failure in the course. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beeper must be Turned off (Proper classroom decorum [behavior] adopts, Course outlines and calendars explain requirements and assignments, students are responsible for knowing what they say. Students are also responsible for doing all assigned work on time. Excessive absences (more than 03) will result in “F Grade”. Students may prepare Sketchbook for taking notes and for references.

Instructor/ Subject Specialist

Chairman/ Head of Department



Institute of Computer Science

Course Outline

COSC-4121 Deep Learning and Applications (3 Credit Hours)

Pre-Requisite: Machine Learning

Program: Computer Science

Total Lectures: 32

Total Tutorials: 0

Instructor: Mr. Shahzad Hussain

Office: 3.2R, ICT Building, KFUEIT

Email: shahzad.hussain@kfueit.edu.pk

Office Hours: 12:00pm -1:30 pm (Wednesday)

Course Introduction

This course provides a comprehensive foundation in deep learning, covering fundamental neural network architectures, optimization strategies, and practical applications. Students will explore Artificial Neural Networks (Perceptron, MLP), Convolutional Neural Networks (CNNs) for Computer Vision, Recurrent Neural Networks (RNN/LSTM) for Sequential Data, Attention Mechanisms, Transfer Learning, and Generative Deep Learning. Hands-on framework practice using PyTorch and TensorFlow is integrated throughout the course to prepare students for real-world artificial intelligence tasks.

Course Contents

1. **Introduction to Artificial Neural Networks & Fundamentals:**

Artificial Neural Networks (Perceptron, MLP), Activation functions (ReLU, Sigmoid, Softmax, Tanh), Binary Classification, Multi-class Classification, PyTorch/TensorFlow framework practice.

2. **Model Training, Optimization & Regularization:**

Backpropagation algorithm, Gradient computation, Advanced optimization methods (SGD, RMSprop, Adam), Advanced regularization techniques (L1/L2 Weight Decay, Dropout).

3. **Foundations of Convolutional Neural Networks (CNNs) for Vision:**

Convolutional Neural Networks (CNNs) for vision, Convnets vs. Densely Connected Networks (MNIST), Core convolution operations (Kernels, feature response maps), Padding (valid vs. same), Strides, Max-Pooling operation.

4. **Small Dataset Training & Data Augmentation:**

Training a Convnet from scratch on small datasets (Dogs vs. Cats classification), Data preprocessing, Image scaling, Directory management, Automated batch processing using ImageDataGenerator, Overfitting diagnosis, Data Augmentation (Rotation, Shift, Shear, Zoom, Flip).

5. **Models & Transfer Learning:**

Transfer Learning principles, Feature Extraction using pretrained networks (VGG16 convolutional base), Freezing layers, Attaching custom dense classifiers, Fine-tuning pretrained CNN models, Architectural overview (VGG16, ResNet50, Inception V3, Xception).

6. **Recurrent Neural Networks & Attention Mechanisms:**

Recurrent Neural Networks (RNN/LSTM) for sequential data, Long-term dependencies, Vanishing gradients, Attention mechanisms, Practical sequential data modeling using PyTorch/TensorFlow.

7. **Generative Deep Learning & DeepDream:**

Introduction to Generative Deep Learning, DeepDream algorithm (Gradient ascent on feature activations), Visualizing network representations, Multi-scale processing (Octaves) using pretrained Inception V3.

Mapping of CLOs and PLOs			
Sr. No	Course Learning Outcomes	PLOs	Blooms Taxonomy
CLO_1	Understand the core concepts, mathematical principles, and architectures of deep learning models.	PLO1	C2 (Understanding)
CLO_2	Analyze and design deep learning solutions for computer vision and sequence modeling applications.	PLO2	C4 (Analyzing)
CLO_3	Develop, train, and evaluate end-to-end deep learning projects using PyTorch/TensorFlow frameworks.	PLO3	C6 (Creating)
Overall Grading Policy			
Assessment Items		Percentage	
Quizzes		5%	
Assignments		5%	
Semester Project		15%	
Midterm Exam		25%	
Final Exam		50%	
Text and Reference Books			
- Ian Goodfellow, Yoshua Bengio, Aaron Courville: <i>Deep Learning</i> (MIT Press) - François Chollet: <i>Deep Learning with Python</i> (Manning Publications) - Aurélien Géron: <i>Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow</i> (O'Reilly)			
Administrative Instructions			
- According to institute policy, 80% attendance is <i>mandatory</i> to appear in the final examination. - Project proposal, SRS, architecture design, and final report must be submitted as per announced format and within due date. - In any case, there will be no retake of (scheduled/surprise) quizzes. - For queries, kindly follow the office hours in order to avoid any inconvenience.			
Computer/LMS Usage			
Students will be required to use Python, PyTorch / TensorFlow, Google Colab, Jupyter Notebooks, and MS Office (Word/PowerPoint) for assignments and semester project submissions.			
Lecture Breakdown			
Week	Topics		
Week 1	Artificial Neural Networks (Perceptron, MLP)		

	• Activation functions (ReLU, Sigmoid, Tanh, Softmax) • Introduction to deep learning workflows
Week 2	• Binary Classification and Multi-class Classification foundations • Loss functions (MSE, Cross-Entropy) • PyTorch/TensorFlow framework practice setup
Week 3	• Backpropagation algorithm and computational graphs • Chain rule and manual gradient computation • Tensor operations and automatic differentiation
Week 4	• Advanced optimization techniques (SGD, Momentum, RMSprop, Adam) • Hyperparameter tuning and learning rate schedulers
Week 5	• Advanced regularization strategies (L1/L2 Weight Decay, Dropout) • Diagnosing overfitting vs. underfitting
Week 6	• Convolutional Neural Networks (CNNs) for vision • Densely Connected Networks vs. Convnets (MNIST Handwritten Digit Classification)
Week 7	• Core convolution operations: Kernels, feature response maps, and output channels • Border effects, Padding (valid vs. same), and Convolution Strides
Week 8	• Max-Pooling operation and feature map downsampling • PyTorch/TensorFlow framework practice for CNNs • Quiz 1
Week 9	Mid Term Exam
Week 10	• Training a Convnet from scratch on small datasets (Dogs vs. Cats classification) • Data preprocessing, image scaling, and directory management • Automated batch processing using ImageDataGenerator
Week 11	• Overfitting diagnosis using training and validation accuracy/loss plots • Mitigating overfitting using Data Augmentation (Rotation, Shift, Shear, Zoom, Flip) and Dropout
Week 12	• Transfer Learning principles and generic feature representations • Feature Extraction using pretrained networks (VGG16 convolutional base)
Week 13	• Freezing layers and attaching custom dense classifiers • Fine-tuning pretrained CNN models on small custom datasets • Architectural overview: VGG16, ResNet50, Inception V3, Xception
Week 14	• Recurrent Neural Networks (RNN/LSTM) for sequential data • Handling long-term dependencies and vanishing gradients in LSTMs
Week 15	• Attention mechanisms and sequence modeling • DeepDream algorithm: Gradient ascent on feature activations • Multi-scale processing (Octaves) and detail reinjection using pretrained Inception V3
Week 16	• Semester Project Presentations and Final Review
Week 17	• Final Exam
Week 18	Final Exam

Khwaja Fareed University of Engineering & Information Technology

PROPOSED TEACHING PLAN

Spring Semester 2022

Department of Computer Science

Teacher's name	Saima Khosa	Email ID	Saima.khosa@kfueit.edu.pk
CourseTitle	Theory of Automata	Program	BS Computer Science,
Course code	COSC-2105	Credit Hours	3
CourseOutline: Finite State Models: Language definitions preliminaries, Formal languages and automata theory, Regular expressions/Regular languages, Finite automata (FAs), Transition graphs (TGs), NFAs, Kleene's theorem, formal modeling, language recognition, introductory logic and formal reasoning, Transducers (automata with output), Pumping lemma and non regular languages. Grammars and PDA: Context free grammars, Derivations, derivation trees and ambiguity, Simplifying CFLs, Normal form grammars and parsing. Context sensitive languages, grammars and linear bounded automata (LBA), Decidability and Undecidability, Chomsky's hierarchy of grammars. Turing Machines Theory: Turing machines, Post machine, Variations on TM, TM encoding, Universal Turing Machine, Defining Computers by TMs, Church-Turing thesis.			

Week	Objective / learning outcomes	Content / sub content	Teaching methodology / activity	Assessment method / technique
1	Automata Theory & Formal Languages Introduction (Chap#1)	Computation, Formal Languages & its Types, What, Why, Where Automata Theory, Real Applications of Automata Theory, Basic constructs: Strings, Words, Alphabet, Language, Languages & Grammars, Basic Language Examples, Formal Modeling, Introductory Logic and Formal Reasoning	Activity based learning	Assignment, Quiz
2	Descriptive Definition of Languages (Chap#2) Recursive Definition of Languages (Chap#3)	Empty/Null String, Valid/Invalid Alphabets, KLEENE Star Closure & KLEENE Plus Closure, Defining Interesting Languages: EQUAL, EVEN-EVEN, INTEGER, factorial, FACTORIAL, DOUBLEFACTORIAL, SQUARE, DOUBLESQUARE, PRIME, PALINDROME etc., Basic Language Recognition Concepts	Activity based learning	Assignment, Quiz
3	Regular Expression (RE) Definition of Languages (Chap#4)	Recursive Definition of RE, RE Basic Constructs, Defining Interesting Languages by using Regular Expression, Language Operators (Union, Concatenation, Star), Operator Precedence, DFA to RE Construction, Algebraic Laws of RE, Some RE language Proofs	Activity based learning	Assignment, Quiz
4	Finite Automata (FA) Definition of Languages (Chap#5)	FA Definition & its basic construct (Transition table, Transition diagrams, start state, final state, transition function, dead state etc.), Defining Interesting Languages by using Finite Automata (starting with specific letters, ending with specific letter, letters occurring specific number of times etc.)	Activity based learning	Assignment, Quiz

Week	Objective / learning outcomes	Content / sub content	Teaching methodology / activity	Assessment method / technique
5	Acceptance, Rejection & Language Recognition cases in DFAs	Acceptance and rejection of strings, Language recognition using DFA, Designing DFAs for specified language conditions, Testing strings for membership, Formal modeling of simple language-recognition problems	Activity based learning	Assignment, Quiz
6	Transition Graph (TG) (Chap#6)	Defining Interesting Languages by using Transition Graph (TG), Generalized Transition Graph (GTG), Defining Interesting Languages by using Generalized Transition Graph (GTG), Basic Introduction to KLEENE's Theorem	Activity based learning	Assignment, Quiz
7	KLEENE's Theorem (Chap#7)	KLEENE's Theorem Part-I, KLEENE's Theorem Part-II, KLEENE's Theorem Part-III, Relationship between regular expressions, finite automata and regular languages	Activity based learning	Assignment, Quiz
8	Non-determinism (Chap#8)	Non-Deterministic Finite Automata (NFA or NDFA), NFA language recognition, Comparing deterministic and non-deterministic acceptance, NFA-based language modeling	Activity based learning	Assignment, Quiz
9	Finite Automata with Output (Chap#9)	Moore Machine, Mealy Machine, Machine Examples, Conversion b/w Machines, Conversion from Moore to Mealy, Conversion from Mealy to Moore	Activity based learning	Assignment, Quiz
10	Regular Languages (Chap#10)	Regular Operations (Union, Concatenation, Kleene Star), Elementary Questions about Regular Languages/Decision properties of Regular Languages, Properties of Regular Languages, Pumping Lemma (Prove a language is not regular), Closure properties of Regular Languages, Equivalence & Minimization of DFA's	Activity based learning	Assignment, Quiz
11	Non-Regular Languages (Chap#11) Context-Free Grammars (Chap#13)	Pumping Lemma, Myhill Nerode Theorem, CFG, Basic Components, Derivations, Ambiguity, Parse Tree etc., Language generation and recognition using CFGs	Activity based learning	Assignment, Quiz
12	Trees (Chap#14)	Basic Tree Structure, Polish Notation, Ambiguous CFG, Regular Grammar Basics, TG for Regular Grammar, Semi-word etc., Null, Nullable, Unit Productions, CNF, Derivation Trees and Parsing	Activity based learning	Assignment, Quiz
13	Push Down Automata (Chap#17)	Push Down Stack, PDA, Non-Deterministic PDA, PDA corresponding to CFG, CFG corresponding to PDA, PDA for Context-Free Language Recognition	Activity based learning	Assignment, Quiz
14	CFG=PDA, Context Free languages	PDA converted form, Equivalence of PDAs & CFG, Converting CFG to PDA & from PDA to CFG, Deterministic PDA (DPDA), PDA converted form, Equivalence of PDAs & CFG, Converting CFG to PDA & from PDA to CFG, Deterministic PDA (DPDA)	Activity based learning	Assignment, Quiz

Week	Objective / learning outcomes	Content / sub content	Teaching methodology / activity	Assessment method / technique
15	(Non-CFL, Parsing, Decidability & Undecidability) Context-Sensitive Languages, Grammars and LBA	Non-Context Free Languages (Chap#20), Non-CFL, Live production, dead production, Theorem, self-embedded non-terminal, Pumping Lemma for CFLs, Context-sensitive languages and grammars, Linear Bounded Automata (LBA), Chomsky's hierarchy of grammars, Decidability, Undecidability and introductory undecidability concepts	Activity based learning	Assignment, Quiz
16	Turing Machines Theory	Turing machines with all its Components, TM language recognition, Post machine, Variations on TM, TM encoding, Universal Turing Machine, Defining Computers by TMs, Church-Turing thesis and the Church-Turing view of computation	Activity based learning	Assignment, Quiz

Recommended Books:

- Daniel I. A. Cohen, "**Introduction to Computer Theory**", Second Edition.
- Sikander H. Khiyal, "**Theory of Automata and Computation**", National Book Foundation, 2004
- Nasir S.F.B and P.K. Srimani, "**A Textbook on Automata Theory**", Cambridge University Press, India, 2008

Additional Reading:

Instructor's Signature



Institute of Computer Science

Course Outline

COSC-3104 Compiler Construction (3 Credit Hours)

Pre-Requisite: Theory of Automata
Program: BS Computer Science
Total Lectures: 32
Total Tutorials: 0
Instructor: Mr. Safdar Hussain
Email: safdar.hussain@kfueit.edu.pk

Course Introduction

This course provides a theoretical and systematic foundation for compiler construction as a formal process of translating programs between languages. Students study compiler design through Formal Languages, regular expressions and finite automata for lexical analysis; context-free grammars and parsing theory for syntax analysis; formal semantics, attribute grammars and type systems for semantic analysis; intermediate representations and syntax-directed translation for intermediate code generation; and program transformation, code optimization and target code generation. The course also develops error handling and recovery techniques, symbol-table management, run-time organization, and formal reasoning about syntax, semantics and semantics-preserving translation.

Course Contents

1. Compiler, interpreter, translator, phases of a compiler, front end and backend, analysis-synthesis model, source/target languages, passes of a compiler, bootstrapping and compiler organization.
2. Alphabets, strings, languages, grammars, formal language definitions, regular languages, context-free languages, Chomsky hierarchy, BNF/EBNF, lexical and syntactic specifications, language recognition and formal modeling of programming languages.
3. Tokens, lexemes and patterns, lexical rules, symbol classes, regular definitions, scanner design, lexical errors, token recognition, buffering and interface with syntax analysis and symbol table.
4. Regular expressions, regular languages, DFA and NFA, transition diagrams, epsilon-NFA, regular expression to finite automata, NFA to DFA conversion, DFA minimization, and use of finite automata in lexical analyzer construction.
5. Context-free grammars, derivations, parse trees, ambiguity, associativity, precedence, left recursion, left factoring, grammar transformation and syntax error detection.
6. Top-Down Parsing, recursive-descent parsing, backtracking, FIRST and FOLLOW sets, Predictive Parsing, LL(1) grammars and parsing-table construction.
7. Shift-reduce parsing, handles and viable prefixes, LR parsing, LR(0) items, SLR(1), CLR/LR(1), LALR(1), parsing tables, shift-reduce/reduce-reduce conflicts and parser generators such as YACC/Bison.
8. Static semantics, syntax-directed definitions and translation, attribute grammars, inherited and synthesized attributes, symbol tables, scopes and bindings, type systems, type expressions, structural and name equivalence, type checking, coercion, and semantic consistency.
9. Abstract syntax trees, DAGs, three-address code, quadruples, triples, control-flow graphs, basic blocks, syntax-directed translation, translation of assignments, Boolean expressions, conditional and loop statements, procedure calls, and backpatching.
10. Activation records, run-time stack, static and dynamic storage, parameter passing, procedure/function calls, non-local references, heap organization and run-time support.
11. Optimization goals and constraints, local and global optimization, constant folding, algebraic simplification, common subexpression elimination, copy propagation, dead-code elimination, code motion, strength reduction, loop optimization, data-flow analysis, basic-block optimization and peephole optimization, with emphasis on semantics preservation.
12. Target-machine model, instruction selection, addressing modes, register allocation and assignment, code generation from intermediate code and DAGs, control-flow translation, instruction ordering and simple machine-dependent optimization.
13. Lexical, syntax and semantic error detection; panic-mode and phrase-level recovery; error reporting; recovery strategies; formal reasoning about language translation; semantic preservation and correctness considerations in compiler transformations.

<i>CLO</i>	<i>Course Learning Outcomes</i>	<i>Bloom's Taxonomy</i>
CLO_1	Explain the theoretical foundations of compiler design using formal languages, regular expressions, finite automata, grammars, parsing theory and formal language specifications.	C2 (Understanding)
CLO_2	Analyze and construct lexical and syntax analyzers using regular expressions, finite automata, context-free grammars, Top-Down Parsing and Predictive Parsing techniques.	C4 (Analyzing)
CLO_3	Design semantic analyses, type-checking systems, intermediate representations, intermediate code generation, code optimization and code generation while applying formal reasoning and semantics-preserving translation.	C6 (Creating)

<i>Assessment Items</i>	<i>Percentage</i>
Quizzes	5%
Assignments	5%
Compiler Implementation Project	15%
Midterm Exam	25%
Final Exam	50%
<i>Text and Reference Books</i>	

- A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd Edition, Addison-Wesley.
- Alfred V. Aho and Jeffrey D. Ullman, Principles of Compiler Design, Addison-Wesley.
- Cooper and Torczon, Engineering a Compiler, 2nd Edition, Morgan Kaufmann.
- Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press.

Computer / Tool Usage

Students may use lexical and parser generator tools such as Lex/Flex and YACC/Bison, along with C/C++ or Python for assignments and the semester compiler project.

Week	Topics
Week 1	• Compiler design overview: compiler, interpreter and translator • Phases of compilation, front end and back end, analysis-synthesis model • Formal translation of source programs and introduction to compiler passes
Week 2	• Formal Languages: alphabets, strings, languages and grammars • Regular languages, context-free languages and Chomsky hierarchy • Formal specification of programming-language syntax and language recognition
Week 3	• Lexical analysis: tokens, lexemes, patterns and regular definitions • Regular expressions for token specification • Lexical errors, scanner design and symbol-table interface
Week 4	• Finite automata for lexical analysis: DFA and NFA • Transition diagrams and epsilon-NFA • RE to FA, NFA to DFA conversion and DFA minimization
Week 5	• Syntax analysis and context-free grammars (CFGs) • Derivations, parse trees and ambiguity • Associativity, precedence, left recursion and left factoring
Week 6	• Top-Down Parsing and recursive-descent parsing • FIRST and FOLLOW sets • Predictive Parsing and LL(1) parsing-table construction
Week 7	• Bottom-up parsing and shift-reduce method • LR(0), SLR(1), LR(1)/CLR and LALR(1) parser theory • Parsing conflicts, parser construction and YACC/Bison overview
Week 8	• Semantic analysis: static semantics and syntax-directed definitions • Attribute grammars, inherited and synthesized attributes • Symbol tables, scopes, bindings and semantic checks
Week 9	Mid Term Exam
Week 10	• Type checking and type systems • Type expressions, structural and name equivalence, type conversion/coercion • Overloading, scope checking and semantic consistency
Week 11	• Intermediate representations: AST, DAG, three-address code • Quadruples, triples, basic blocks and control-flow graphs • Intermediate code generation for assignments and expressions
Week 12	• Syntax-directed translation for Boolean expressions and control flow • Backpatching for conditional and loop statements • Intermediate code generation for procedures and function calls
Week 13	• Run-time organization: activation records, stacks and heaps • Parameter passing and procedure calls • Storage allocation and run-time support for translated programs
Week 14	• Code optimization: goals, constraints and semantics preservation • Local and global optimizations: constant folding, common subexpressions, copy propagation and dead-code elimination • Loop optimizations, data-flow analysis and basic-block optimization
Week 15	• Code generation and target-machine model • Instruction selection, addressing modes and register allocation • Code generation from intermediate code/DAGs and peephole optimization
Week 16	• Error handling and recovery: lexical, syntax and semantic errors • Panic-mode and phrase-level recovery, error reporting and recovery strategies • Formal reasoning about translation and compiler correctness
Week 17	• Semester Project Presentations and Final Review • Integrated review: lexical analysis to code generation • Discussion of semantics-preserving translation, optimization and implementation issues
Week 18	Final Exam



Tentative Weekly Course Plan

Course Title	Linear Algebra		
Course Code	MATH-2103	Credit Hour	3
Name	Mr. Ubaid-ur-Rehman Asghar	Email	ubaid.rehman@kfueit.edu.pk

Course Contents (Core Topics):

- Systems of Linear Equations and Matrices: Gaussian Elimination, Matrix Inverses, Determinants (with proof of properties).
- Vector Spaces and Subspaces: Formal definition of Vector Spaces, Subspaces, Linear Span, Linear Independence and Dependence (with proofs of theorems).
- Basis and Dimension: Formal proof of Existence of Basis, Change of Basis, Rank and Nullity Theorem.
- Linear Transformations: Formal definition and properties of Linear Transformations, Kernel and Range (with proofs).
- Eigenvalues and Eigenvectors: Calculation and theoretical properties of Eigenvalues and Eigenvectors, Diagonalization.
- Inner Product Spaces: Orthogonalization (Gram-Schmidt process) and Projection.

Resource Material	1. Reference Books	
	i	Linear Algebra and Its Applications by <i>David C. Lay</i> or <i>Gilbert Strang</i>
	ii	Elementary Linear Algebra (Applications Version) by Howard Anton
	2. Web Resources	
	i.	https://3blue1brown.com/topics/linear-algebra
	ii.	https://khanacademy.org/math/linear-algebra

TOPICS TO BE COVERED

Week #	Topics (outline of main topics and sub topics)	Tutorial /Laboratory
1	Real and complex numbers, Introduction to Matrices and Systems of Equations	Tutorial
2	Gaussian Elimination, Gauss-Jordan Elimination, Matrix Operations	Tutorial
3	Determinants and their Theoretical Properties (Proof of basic theorems)	Tutorial
4	Formal Definition of Vector Spaces and their Subspaces	Tutorial



5	Linear Combination, Span, and Proof of Linear Independence/Dependence	Tutorial
6	Basis and Dimension of a Vector Space	Tutorial
7	Row Space, Column Space, and Null Space	Tutorial
8	Linear Transformations and Linear Mappings	Tutorial

9	Definition, Matrix Representation, Properties (Kernel and Range)	
Mid Term Exam		
10	Inner Product Spaces, Orthogonality, Orthogonal Complements	Tutorial
11	Gram-Schmidt Orthogonalization Process	Tutorial
12	Eigenvalues and Eigenvectors (Calculation and Geometric Interpretation)	Tutorial
13	Diagonalization and its applications	Tutorial
14	Mathematical reasoning and Applications of Linear Algebra relevant to Computer Science	Tutorial
15	Review and Introduction to Symmetric Matrices	Tutorial
16	Quadratic Forms (connecting to optimization)	
Final Term Exam		

Student Evaluation criteria:

Assignments	5%
Surprise Test, Quizzes	5%
Presentation	10%
Mid Term Paper	30%
Final Term paper (Marks Distribution: 20 % from Mid Term and 80% from Final Term), i.e. 10 + 40=50	50%
Total	100%

Student Responsibilities:

A student must have attended at least **75%** of the classes held in a course in order to be allowed to sit in the final examination. Students must also arrive on time and remain in class for the entire period. Cellular Phones and Beeper must be Turned off (Proper classroom decorum [behavior] adopts, Course outlines and calendars explain requirements and assignments. Students are also responsible for doing all assigned work on time. Students may prepare Sketchbook for taking notes and for references.



Course Outline

MATH-1100 CALCULUS (3 Credit Hours)

Pre-Requisite:

Course Title: Calculus

Code: MATH-1100

Credit Hours: 3

Instructor: Mr. Nasir Nadeem

Office: Mechanical Engineering Department, KFUEIT

Email: nasir.nadeem@kfueit.edu.pk

Office Hours: 10:30am - 12:30 pm

Course Objective

This foundational course provides rigorous study in Mathematical Analysis, focusing on theoretical concepts, formal definitions, and proof techniques essential for higher-level quantitative studies, including an introduction to multivariate functions

Course Contents

Foundations of Analysis: Limits, continuity, sequences, and series with emphasis on $\epsilon - \delta$ proofs (to satisfy the 'proofs' requirement).

Differential Calculus: Functions and Inverse Functions, Techniques of differentiation, Mean Value Theorem, Taylor/Maclaurin Series.

Vector and Multivariate Calculus: Functions of several variables, Partial Derivatives (First and Second Order), Multiple Integrals (Double and Triple), and basic concepts of Vector Fields.

Mapping of CLOs and PLOs

Sr. No	Course Learning Outcomes	PLOs	Blooms Taxonomy
1	Students will be able to apply formal mathematical proof techniques to demonstrate the properties of limits and continuity	-	-
2	Students will be able to differentiate and integrate functions of several variables, demonstrating proficiency in Partial Derivatives and Multiple Integration	-	-
3	Students will analyze complex problems using Taylor/Maclaurin Series and apply fundamental theorems of calculus to various domains	-	-

CLO Assessment Mechanism

Assessment tools	CLO_1	CLO_2	CLO_3
Quizzes / Assignments	-	-	-
Mid semester Exam	-	-	-
Final Exam	-	-	-

Overall Grading Policy

Assessment Items	Percentage
Sessional (Quizzes + Assignments + ppts)	20%
Midterm Exam	30%

Final Exam		50%
Text and Reference Books		
Text books: - Calculus: Early Transcendentals by <i>James Stewart</i> - Thomas' Calculus by <i>George B. Thomas, Jr.</i> et al Reference books: - Introduction to Real Analysis by <i>Robert G. Bartle</i>		
Administrative Instructions		
- According to institute policy, 75% attendance is <i>mandatory</i> to appear in the final examination. - Assignments and report must be submitted as per announced format and within due date. - For queries, kindly follow the office hours in order to avoid any inconvenience.		
Lecture Breakdown		
Week 01	Introduction to Real Numbers, Functions, Inverse Functions, Definition of Limits ($\epsilon \rightarrow \delta$)	
Week 02	Continuity, Inverse Functions, Differential Calculus, Basic Rules of Differentiation	
Week 03	Chain Rule, Implicit Differentiation, Related Rates	
Week 04	Theorems of Differentiation (Rolle's Theorem, Mean Value Theorem, L'Hôpital's Rule)	
Week 05	Integration and Integral Calculus, Definite and Indefinite Integrals, Fundamental Theorem of Calculus.	
Week 06	Techniques of Integration (Substitution, By Parts)	
Week 07	Applications of Integrals (Area, Volume)	
Week 08	Sequences and Infinite Series, Convergence and Divergence Tests, Taylor Expansion	
Week 09	Mid Term Exam	
Week 10	Introduction to Multivariate Functions	
Week 11	Level Curves and Limits	
Week 12	Continuity in Higher Dimensions	
Week 13	Partial Derivatives (First Order), Chain Rule for Multiple Variables	
Week 14	Second Order Partial Derivatives, Mixed Partial Derivatives, Critical Points	
Week 15	Multiple Integrals (Double Integrals over Rectangular and General Regions)	
Week 16	Change of Variables in Multiple Integrals (Polar, Cylindrical coordinates)	
Week 17	Introduction to Vector Fields, Applications of Calculus to Mathematical and Computational Problems	
Week 18	Final Term Exam	



Probability and Statistics

Credit Hours:

3

Course Code

STAT-2104

Instructor

Mr. Muhammad Hashir
Zubair

Objectives:

This course provides a mathematical and analytical foundation in probability and statistics, with emphasis on their theoretical principles and practical applications. It develops rigorous understanding of uncertainty, random phenomena, statistical modeling, estimation, inference, and regression, with emphasis on concepts used in data analysis.

Course Learning Outcomes:

- Students will understand fundamental principles of probability theory, statistical reasoning, and uncertainty modeling.
- Students will formulate and analyze random variables, probability distributions, expectations, variances, and joint distributions.
- Students will apply Bayes' theorem, conditional probability, and independence to probabilistic reasoning and data-driven problems.
- Students will perform descriptive and inferential statistical analysis, including estimation, confidence intervals, and hypothesis testing.
- Students will develop and interpret regression models and apply statistical methods to problems in CS.

Course Contents:

- Introduction to Probability and Statistics: populations and samples, variables, data types, measurement scales, statistical reasoning, and applications in Computer Science.
- Descriptive Statistics: measures of central tendency, dispersion, quartiles, percentiles, skewness, kurtosis, frequency distributions, and graphical summaries.
- Probability Theory: sample spaces, events, set operations, counting principles, permutations, combinations, axioms of probability, and basic probability rules.
- Conditional Probability, Independence, and Bayes' Theorem: conditional probability, multiplication rule, law of total probability, independence, Bayes' theorem, and Bayesian reasoning.
- Random Variables: discrete and continuous random variables, probability mass functions, probability density functions, cumulative distribution functions, and transformations.
- Probability Distributions: Bernoulli, Binomial, Geometric, Negative Binomial, Hypergeometric, Poisson, Uniform,

Exponential, Gamma, Normal, and related distributions.

- Expectation and Variance: expected value, moments, variance, standard deviation, linearity of expectation, conditional expectation, variance rules, and moment-generating concepts.
- Joint Random Variables: joint, marginal, and conditional distributions, covariance, correlation, independence, conditional expectation, and functions of random variables.
- Sampling and Limit Results: sampling distributions, Law of Large Numbers, Central Limit Theorem, normal approximation, and statistical foundations for large datasets.
- Statistical Estimation: point estimation, estimators, bias, consistency, efficiency, maximum likelihood estimation, least-squares estimation, and sampling distributions of estimators.
- Interval Estimation: confidence intervals for means, proportions, variances, and regression parameters; interpretation and uncertainty quantification.
- Statistical Analysis and Inference: null and alternative hypotheses, test statistics, p-values, significance levels, Type I and Type II errors, statistical power, z-tests, t-tests, chi-square tests, and basic ANOVA.
- Regression Analysis: simple and multiple linear regression, least-squares fitting, model parameters, residual analysis, coefficient interpretation, prediction, confidence intervals, and model assumptions.
- Statistical Modeling : correlation analysis, model diagnostics, uncertainty in predictions, feature relationships, statistical decision-making, and interpretation of empirical results.

Text Books:

Sheldon M. Ross, A First Course in Probability

Ronald E. Walpole et al., Probability and Statistics for Engineers and Scientists

Recommended Books:

George Casella and Roger L. Berger, Statistical Inference

Douglas C. Montgomery and George C. Runger, Applied Statistics and Probability for Engineers

Weekly Plan

Sr. No	Weekly	Topics
1	Week 1st	Introduction to Probability and Statistics; populations, samples, variables, data types, descriptive statistics, measures of central tendency and dispersion
2	Week 2nd	Probability theory; sample spaces, events, set operations, counting principles, permutations, combinations, axioms and basic probability rules
3	Week 3rd	Conditional probability, independence, law of total probability, and Bayes' theorem; probabilistic reasoning and applications
4	Week 4th	Random variables; discrete and continuous random variables, PMF, PDF, CDF, transformations, and probability calculations
5	Week 5th	Probability distributions I; Bernoulli, Binomial, Geometric, Negative Binomial, Hypergeometric, and Poisson distributions
6	Week 6th	Probability distributions II; Uniform, Exponential, Gamma, Normal distributions; standardization and distributional relationships
7	Week 7th	Expectation and variance; moments, linearity of expectation, conditional expectation, covariance, correlation, and joint random variables
8	Week 8th	Sampling distributions; Law of Large Numbers, Central Limit Theorem, normal approximation, and mid-course revision
9	Week 9th	Mid Term Exam
10	Week 10th	Statistical estimation; point estimators, bias, consistency, efficiency, maximum likelihood estimation, least-squares estimation
11	Week 11th	Interval estimation; confidence intervals for means, proportions, variances, and regression parameters; interpretation of uncertainty
12	Week 12th	Statistical analysis and inference; hypothesis testing, p-values, significance levels, Type I/II errors, power, z-tests and t-tests
13	Week 13th	Chi-square tests, goodness-of-fit, independence tests, one-way ANOVA, and comparison of statistical groups
14	Week 14th	Regression analysis I; simple linear regression, least squares, coefficient estimation, correlation, residuals, assumptions, and prediction
15	Week 15th	Regression analysis II and applications; multiple linear regression, model diagnostics, confidence intervals
16	Week 16th	Final revision, integrated problem solving
17	Week 17th	Statistical interpretation, and evaluation
18	Week 18th	Final Exam



Course Outline

MATH-2122 Differential Equations (3 Credit Hours)

Pre-Requisite: Calculus

Course Title: Differential Equations

Course Code: MATH-2122

Credit Hours: 3

Instructor: Ms. Saba Jamil

Office: 2.21DR, 1st Floor, Electrical Engineering Building, KFUEIT, RYK

Email: saba.jamil@kfueit.edu.pk

Office Hours: 13:30 - 14:30

Course Objective

This course extends the concepts of Calculus by introducing advanced analytical techniques to model and solve real-world problems. It covers higher-dimensional systems and theoretical foundations essential for understanding complex mathematical models

Course Contents

First Order ODEs: Analytical solutions (Separable, Exact, Linear).

Higher Order ODEs: Homogeneous and non-homogeneous equations.

Vector and Higher-Dimensional Systems: Systems of Linear Differential Equations, Matrix methods for solving systems (proofs of existence and uniqueness of solutions are a key focus).

Partial Differential Equations (PDEs): Introduction to first-order PDEs, boundary value problems, and Fourier Series.

Mapping of CLOs and PLOs

Sr. No	Course Learning Outcomes	PLOs	Blooms Taxonomy
1	Students will be able to apply advanced analytical methods to solve ordinary and systems of differential equations	-	-
2	Students will develop the ability to theoretically analyze the behavior of solutions, including stability and uniqueness (demonstrating analysis with proofs)	-	-
3	Students will be able to formulate simple real-world models using differential equations, including an introduction to higher-dimensional modeling	-	-

CLO Assessment Mechanism

Assessment tools	CLO_1	CLO_2	CLO_3
Quizzes / Assignments	-	-	-
Mid semester Exam	-	-	-
Final Exam	-	-	-

Overall Grading Policy

Assessment Items	Percentage
Sessional (Quizzes + Assignments + ppts)	20%
Midterm Exam	30%
Final Exam	50%

Text and Reference Books

- **Text book:** Elementary Differential Equations and Boundary Value Problems by *Boyce and DiPrima*
- **Reference book:** Differential Equations by *Zill*

Administrative Instructions

- According to institute policy, 75% attendance is *mandatory* to appear in the final examination.
- Assignments and report must be submitted as per announced format and within due date.
- For queries, kindly follow the office hours in order to avoid any inconvenience.

Lecture Breakdown

Week 01	First-order ordinary differential equations (ODEs), Classification, Separable equations, solution methods, and First-Order Linear Equations (Integrating Factors)
Week 02	Solutions to First Order Linear Equations (Integrating Factors)
Week 03	Exact Equations and Numerical Methods for First Order ODEs
Week 04	Second-order linear differential equations, Homogeneous and non-homogeneous equations
Week 05	Method of Undetermined Coefficients and Variation of Parameters
Week 06	Series Solutions for ODEs (Taylor and Power Series)
Week 07	Laplace transforms and their application to solving differential equations
Week 08	Systems of Linear Differential Equations, Matrix Formulation
Week 09	Eigenvalues and Eigenvectors for systems of ODEs
Mid Term	
Week 10	Phase Plane Analysis and Stability of systems
Week 11	Introduction to Partial Differential Equations (PDEs), Classification
Week 12	Solutions of simple PDEs (Separation of Variables)
Week 13	Fourier Series and Boundary Value Problems
Week 14	Applications of Differential Equations in Complex Modeling and Data Analysis
Week 15	Numerical Methods for Solving Differential Equations and Error Analysis
Week 16	Presentations

Final Term



Course Outline

MATH-2124 Numerical Analysis (3 Credit Hours)

Course Title: Numerical Analysis
Course Code: MATH-2124
Credit Hours: 3
Instructor: Ms. Huba Rana
Office: Mechanical Engineering Department, KFUEIT
Email: huba.rana@kfueit.edu.pk
Office Hours: 14:30am - 15:30 pm (Tuesday)

Course Objective

This course is to introduce students to the mathematical concepts and computational techniques required to find approximate solutions to complex mathematical problems that lack analytical (exact) solutions. The course emphasizes understanding the algorithms, error analysis, and stability of numerical methods

Mapping of CLOs and PLOs

Sr. No	Course Learning Outcomes	PLOs	Blooms Taxonomy
1	Apply and analyze various numerical methods (e.g., Bisection, Newton-Raphson) to efficiently find the roots of non-linear equations.	PLO-1	C3 (Applying) / C4 (Analyzing)
2	Solve systems of linear equations using direct (e.g., Gaussian Elimination) and iterative (e.g., Jacobi, Gauss-Seidel) techniques and understand their computational efficiency	PLO-2	C3 (Applying)
3	Use techniques like Lagrange and Newton interpolation to approximate complex functions, and apply the concept of Least Squares Approximation to fit data (fundamental in Machine Learning).	PLO-2	C3 (Applying)
4	Implement numerical methods for integration (Quadrature) and differentiation, and apply methods for solving Ordinary Differential Equations (ODEs) like the Runge-Kutta method.	PLO-1	C3 (Applying)

CLO Assessment Mechanism

Assessment tools	CLO_1	CLO_2	CLO_3
Quizzes / Assignments	-	-	-
Mid semester Exam	-	-	-
Final Exam	-	-	-

Overall Grading Policy

Assessment Items	Percentage
Sessional (Quizzes + Assignments + ppts)	25%
Midterm Exam	25%
Final Exam	50%

Text and Reference Books

Recommended books:

- Numerical Analysis by *Richard L. Burden and J. Douglas Faires*
- Numerical Methods for Engineers by *Steven C. Chapra and Raymond P. Canale*
- Introduction to Numerical Analysis by *S. S. Sastry*

Administrative Instructions

- According to institute policy, 75% attendance is *mandatory* to appear in the final examination.
- Assignments and report must be submitted as per announced format and within due date.
- For queries, kindly follow the office hours in order to avoid any inconvenience.

Lecture Breakdown

Week 01-02	Mathematical preliminaries, Sources of Errors (Round-off, Truncation), Absolute and Relative Errors, Stability and Conditioning of problems
Week 03-04	Solutions of non-linear equations (Bisection, Newton-Raphson, Fixed-Point Iteration, Secant Method), Convergence Rates
Week 05-06	Direct & Iterative linear systems (Gauss Elimination, LU Decomposition)
Week 07-08	Direct & Iterative linear systems (Jacobi Method, Gauss-Seidel Method, SOR, Convergence Criteria)
Week 09-10	Lagrange Interpolating Polynomials, Divided Differences, Newton's Form of the Interpolating Polynomial
Week 11	Numerical Differentiation & Integration (Finite Differences, Trapezoidal Rule)
Week 12-13	Numerical Differentiation & Integration (Simpson's Rules, Composite Rules, Gaussian Quadrature)
Week 14	Polynomial Approximation, Linear Least Squares Regression, Non-linear Least Squares
Week 15	Numerical ODEs: Initial Value Problems, Euler's Method
Week 16	Runge-Kutta Methods (RK2, RK4), and Multi-Step Methods



Discrete Structures					
Credit Hours:	3	Course Code	COSC-1103	Instructor	Mr. Mujeeb Ur Rehman
Objectives:					
This course introduces the fundamental mathematical and abstract structures used in Computer Science and Data Science. It develops logical reasoning, proof techniques, discrete problem-solving skills, combinatorial methods, and the study of sets, relations, functions, number theory, graphs, Boolean algebra, coding theory, and cryptographic foundations essential for algorithmic analysis and theoretical computer science.					
Course Learning Outcomes:					
CLO 1: Students will be able to apply mathematical logic, set theory, relations, functions, and formal proof techniques, including direct proof, contradiction, and mathematical induction, to solve discrete problems. CLO 2: Students will be able to analyze and solve discrete mathematical problems using combinatorics, recurrence relations, number theory, Boolean algebra, and related mathematical structures. CLO 3: Students will be able to model and analyze computational problems using graph theory, trees, relations, and other discrete structures, including fundamental graph properties and algorithms. CLO 4: Students will be able to apply concepts of coding theory and cryptographic foundations, along with discrete mathematical reasoning, to basic computer science and data-oriented problems.					
Course Contents:					
Sets, relations and functions; propositional and predicate logic, logical equivalence and proof techniques including induction, contradiction and direct proof; counting principles, permutations, combinations, pigeonhole principle, inclusion-exclusion and recurrence relations; sequences and summation; number theory including prime numbers, divisibility and modular arithmetic; Boolean algebra and logic gates; graph theory, trees, graph traversal, paths, circuits, Eulerian and Hamiltonian graphs, graph coloring and basic graph properties; discrete mathematical problem solving; coding theory and cryptographic foundations.					
Text Books:					
1. Discrete Mathematics and Its Applications by <i>Kenneth H. Rosen</i>					
Recommended Books:					
1. Discrete Mathematics by <i>Richard Johnsonbaugh</i>					

Weekly Plan of “Discrete Structures” by Mr. Mujeeb Ur Rehman

Sr. No	Weekly	Topics
1	Week 1st	Introduction to Propositional Logic and Compound Propositions,
2	Week 2nd	Conditional Statements, Logical Equivalence
3	Week 3rd	Methods of Proof (Direct Proof, Contraposition, Contradiction)
4	Week 4th	Mathematical Induction
5	Week 5th	Set Theory, Set Operations, Set Identities (with proofs)
6	Week 6th	Functions, Sequences, Summation Notation, and Introduction to Recurrence Relations
7	Week 7th	Relations, Equivalence Relations, Partial Orderings
8	Week 8th	Number Theory: Prime Numbers, Modular Arithmetic, and Discrete Mathematical Problem Solving
Mid Term Exam		
9	Week 9th	Combinatorial Reasoning: Principles of Counting, Pigeonhole Principle, Inclusion-Exclusion
10	Week 10th	Permutations, and Combinations
11	Week 11th	Introduction to Graph Theory, Terminology
12	Week 12th	Isomorphism, Trees
13	Week 13th	Eulerian and Hamiltonian Paths, Graph Coloring, Shortest Path Algorithms
14	Week 14th	Boolean Algebra and Logic Gates
15	Week 15th	Coding Theory and Cryptographic Foundations (Applications of Discrete Mathematics in Security)
16	Week 16th	Presentation
Final Term Exam		



Course description

Final Year Project - I and II (6 Credit hours)

Course Title: Final Year Project - I and II

Course Code: COSC-4301 and COSC-4302

Credit Hours: 3 + 3

Instructor: Mr. Mutiullah

Office: 3.2L, 3rd Floor, Institute of Computer Science, ICT Building, KFUEIT, RYK

Email: mutiullah.jamil@kfueit.edu.pk

Office Hours: 10:30 - 11:30

Course Objective

This course provides a structured two-phase framework for the development of a final year project. The first phase focuses on problem identification, literature review, requirements analysis, feasibility assessment, project planning, system design, and preparation of technical documentation. The second phase focuses on implementing the proposed solution through programming, frameworks, module development, system integration, testing, debugging, and performance improvement. Throughout both phases, students work under the guidance of their supervisor through regular progress meetings and conclude the project with formal documentation, presentation, and evaluation before the project panel.

Course Contents

- Identification and analysis of a relevant problem, project scope, objectives, and expected outcomes.
- Literature review and analysis of existing solutions, technologies, and methodologies.
- Requirements gathering, feasibility analysis, development methodology, and project planning.
- Preparation of Software Requirements Specification (SRS), system architecture, database design, interface design, and other technical documentation.
- Development and implementation of the proposed system using appropriate programming languages, frameworks, tools, and technologies.
- Implementation and integration of project modules and system components.
- Testing, debugging, security validation, performance improvement, and system refinement.
- Preparation of user manual, final technical documentation, project report, and presentation.
- Final project demonstration, Viva/Question-Answer Session, and evaluation by the project panel.

Mapping of CLOs and PLOs

Sr. No	Course Learning Outcomes	PLOs	Blooms Taxonomy
1	Students will be able to analyze a problem, define project requirements, objectives, scope, and appropriate development methodology.	-	-

2	Students will be able to design and document the proposed system, including its architecture, modules, database, and user interface.	-	-
3	Students will be able to implement, integrate, test, debug, and improve the proposed system using appropriate technologies and development frameworks.	-	-
4	Students will be able to prepare and present the complete project documentation, demonstrate the developed system, and defend the project professionally before the evaluation panel.	-	-

CLO Assessment Mechanism

Assessment Tools	CLO_1	CLO_2	CLO_3	CLO_4
Supervisor Evaluation (Meetings, Progress, Implementation)	✓	✓	✓	-
Final Presentation & Demonstration	✓	✓	✓	✓
Viva / Question-Answer Session	-	-	✓	✓

Overall Grading Policy

Assessment Items	Percentage
Supervisor Evaluation (Progress, Meetings, Implementation)	30%
Final Presentation & Viva (Evaluation by Panel)	70%

Administrative Instructions

- Students must maintain regular contact with their supervisor and submit progress reports on time.
- The project must be implemented practically with working functionality and proper source code.
- All deadlines for prototype submission, documentation, and final demo must be strictly followed.
- Attendance and participation in project meetings are mandatory as per institute policy.
- Students must be prepared for final viva, demonstration, and question-answer session.

Project Breakdown

Week	Topics
Week 01	Project orientation, idea identification, and problem definition
Week 02	Project scope, objectives, and expected outcomes
Week 03	Background study and literature review
Week 04	Analysis of existing systems and proposed solution
Week 05	Requirements gathering and functional requirements
Week 06	Non-functional requirements and project constraints
Week 07	Feasibility analysis and development methodology

Week 08	Technology selection and project development planning
Week 09	System architecture and high-level system design
Week 10	Database design and module/component design
Week 11	User interface / prototype design and workflow definition
Week 12	Software Requirements Specification (SRS) preparation
Week 13	Project report and technical documentation preparation
Week 14	Documentation review, refinement, and supervisor feedback
Week 15	Final documentation and presentation preparation
Week 16	FYP-I Final Presentation, Viva, and Panel Evaluation
Week 17	FYP-II implementation planning and development environment setup
Week 18	Core functionality development - Part 1
Week 19	Core functionality development - Part 2
Week 20	Module development and implementation
Week 21	Database integration and backend/system integration
Week 22	User interface implementation and refinement
Week 23	Integration of all major modules
Week 24	Initial system testing and validation
Week 25	Debugging, error handling, and issue resolution
Week 26	Additional feature development and system optimization
Week 27	Security validation and performance enhancement
Week 28	Comprehensive system testing and refinement
Week 29	User/manual documentation and final technical documentation
Week 30	Final code cleanup, system testing, and project completion
Week 31	Final project rehearsal and presentation preparation
Week 32	Final Demonstration, Viva, Panel Evaluation, and Submission